

Sprint 9 Structured Notes: Collaboration, Forks, Sync, `.gitignore`, Pages, and Conflicts

1. Current Progress

You are now at **Sprint 9** in the Relational Databases learning path.

So far, you have completed:

- **Sprint 1:** VS Code and workspace setup
- **Sprint 2:** Terminal, shell, and command line basics
- **Sprint 3:** Bash file and folder commands
- **Sprint 4:** Bash scripting basics
- **Sprint 5:** Nano and Git setup
- **Sprint 6:** Local Git workflow
- **Sprint 7:** GitHub repositories and remotes
- **Sprint 8:** Branches, pull requests, and code review

You are still inside the **Git section** of the course. You are not learning PostgreSQL or SQL yet.

In Sprint 8, you learned the normal team workflow:

```
main branch → feature branch → commit → push → pull request → review → merge
```

Now Sprint 9 goes one level deeper. It teaches what happens when you work with **other people's repositories**, keep your work synchronized, ignore private files, publish a small site, and fix merge conflicts.

2. Main Goal of Sprint 9

The main goal is:

Handle common GitHub collaboration tasks safely.

In simple words:

You will learn how to work with projects that are not only yours. You will also learn how to avoid uploading private files, deploy a simple site, and fix conflicts when Git cannot automatically combine changes.

Sprint 9 covers these ideas:

- Forking

- Write access
 - `origin` vs `upstream`
 - Contributor workflow
 - Syncing with the original repository
 - `.gitignore`
 - GitHub Pages
 - Merge conflicts
 - Conflict markers
 - Rebase
 - Interactive rebase
 - Squash
-

3. Big Beginner Idea

Imagine GitHub like this:

- **Your own repository** = your notebook
- **Someone else's repository** = teacher's notebook
- **Fork** = your copy of teacher's notebook
- **Pull request** = asking the teacher to accept your changes
- **Sync** = updating your notebook with the teacher's latest changes
- **Conflict** = both people changed the same sentence, so Git asks you to choose

Sprint 9 teaches you how to work safely in this situation.

Part 1: Forking

4. What Is a Fork?

A **fork** is your own copy of someone else's GitHub repository.

You use a fork when you do not have permission to directly change the original repository.

Example:

Original repository:

teacher/project

Your fork:

```
your-name/project
```

Simple meaning:

Fork = my GitHub copy of someone else's project.

5. Why Do We Fork?

You fork when:

- You want to contribute to an open-source project.
- You do not have write access to the original repository.
- You want to safely experiment with someone else's project.
- You want to send changes using a pull request.

Simple meaning:

Forking lets you work on another person's project without changing their original project directly.

Part 2: Write Access

6. What Is Write Access?

Write access means permission to push changes directly to a repository.

If you have write access, you can push branches to that repository.

If you do not have write access, you usually fork the repository first.

Simple meaning:

Write access means you are allowed to upload changes to that repository.

Example:

If this is your own repo:

```
your-name/sprint9-practice
```

You probably have write access.

If this is another person's repo:

```
company/project
```

You may not have write access.

Then you use a fork.

Part 3: origin vs upstream

7. What Is origin?

`origin` is usually the default name for your main remote repository.

In Sprint 7, you learned that `origin` normally points to your GitHub repository.

Example:

```
git remote -v
```

Output idea:

```
origin git@github.com:your-name/project.git
```

Simple meaning:

origin = my online repository.

8. What Is upstream?

`upstream` usually means the original repository that your fork came from.

Example:

Original project:

```
teacher/project
```

Your fork:

```
your-name/project
```

In your local computer:

```
origin    = your fork  
upstream  = original project
```

Simple meaning:

upstream = the original repository.

9. Adding an Upstream Remote

If you cloned your fork, you can add the original repository as `upstream`.

Command:

```
git remote add upstream <original-repository-url>
```

Example:

```
git remote add upstream git@github.com:teacher/project.git
```

Then check:

```
git remote -v
```

You should see something like:

```
origin    git@github.com:your-name/project.git  
upstream  git@github.com:teacher/project.git
```

Simple meaning:

Now Git knows both your copy and the original project.

Part 4: Open-Source Contributor Workflow

10. Basic Contributor Flow

When contributing to another project, the common flow is:

```
Fork original repo
↓
Clone your fork
↓
Add upstream
↓
Create a branch
↓
Make changes
↓
Commit
↓
Push to your fork
↓
Open pull request to original repo
```

Simple meaning:

You work in your copy, then ask the original project owner to accept your changes.

11. Contributing Guidelines

Many projects have a file such as:

```
CONTRIBUTING.md
```

This file explains how to contribute.

It may tell you:

- How to name branches
- How to write commits
- How to run tests
- How to open pull requests
- What coding style to follow
- Who reviews the work

Simple meaning:

Contributing guidelines are the project rules for helpers.

Before contributing to a real project, always read:

```
README.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
```

Part 5: Syncing with Upstream

12. Why Syncing Is Needed

Imagine you forked a project last week.

Since then, the original project changed.

Your fork is now old.

So you need to bring the latest original changes into your local project.

Simple meaning:

Syncing means updating your copy with the latest original project changes.

13. `fetch` vs `pull`

```
git fetch
```

```
git fetch upstream
```

This downloads information from the remote, but it does not automatically change your files.

Simple meaning:

fetch = check and download remote history, but do not merge yet.

```
git pull
```

```
git pull
```

This downloads and also tries to merge changes into your current branch.

Simple meaning:

pull = fetch + merge.

Beginner memory:

```
fetch = bring information  
pull = bring information and apply it
```

14. Safe Sync from Upstream

A common safe flow is:

```
git checkout main  
git fetch upstream  
git merge upstream/main
```

Meaning:

```
git checkout main
```

Go to your main branch.

```
git fetch upstream
```

Download the latest information from the original repository.

```
git merge upstream/main
```

Merge the original main branch into your local main branch.

Simple meaning:

This updates your local main branch with the original project's latest work.

Part 6: Hard Reset and Force Push

15. What Is `git reset --hard`?

Command:

```
git reset --hard upstream/main
```

This makes your current branch exactly match `upstream/main`.

But it can delete local changes that are not saved elsewhere.

Simple meaning:

Hard reset is like saying, "Throw away my current local state and copy that branch exactly."

Be very careful.

Use it only when you understand what you are losing.

16. What Is `git push --force`?

Command:

```
git push --force
```

This forces GitHub to accept your local history, even if it rewrites remote history.

Simple meaning:

Force push can overwrite remote history.

This is dangerous in team projects because it can damage other people's work.

Beginner rule:

Do not use:

```
git reset --hard
git push --force
```

unless your teacher, team lead, or mentor clearly tells you to do it.

Part 7: `.gitignore`

17. What Is `.gitignore`?

`.gitignore` is a file that tells Git:

```
Do not track these files or folders.
```

Simple meaning:

`.gitignore` protects your repository from unwanted files.

Examples of files and folders to ignore:

```
.env
node_modules/
dist/
.vscode/
.DS_STORE
Thumbs.db
```

18. Why `.gitignore` Is Important

You should ignore:

- Secret files
- Password files
- API keys
- Dependency folders
- Build output folders
- IDE setting folders
- Operating system files

Example:

```
.env
```

This file may contain passwords or secret keys.

Do not push it to GitHub.

19. Example `.gitignore` File

Create a file named:

```
.gitignore
```

Add:

```
.env  
github.key  
node_modules/  
dist/  
coverage/  
.vscode/  
!.vscode/settings.json  
.DS_STORE  
Thumbs.db
```

Meaning:

```
.env
```

Ignore environment secret files.

```
node_modules/
```

Ignore dependency folder.

```
dist/
```

Ignore build output.

```
.vscode/
```

Ignore VS Code settings folder.

```
!.vscode/settings.json
```

Exception: allow this one file.

```
.DS_STORE  
Thumbs.db
```

Ignore system files created by macOS or Windows.

20. Important `.gitignore` Warning

If a file was already committed before, adding it to `.gitignore` does not fully remove it from Git history.

Example:

You already committed:

```
.env
```

Then later you add `.env` to `.gitignore`.

Git may still remember the old committed file.

Simple meaning:

`.gitignore` prevents future tracking, but it does not automatically erase old committed history.

Part 8: GitHub Pages

21. What Is GitHub Pages?

GitHub Pages lets you publish a static website from a GitHub repository.

A static website can include:

```
HTML
CSS
JavaScript
Images
Documentation pages
```

Simple meaning:

GitHub Pages lets you turn a GitHub repo into a simple website.

22. Basic GitHub Pages Flow

For a simple HTML project:

Create or use a repository with:

```
index.html
```

Push it to GitHub.

Go to GitHub repository:

```
Settings → Pages
```

Choose:

```
Deploy from a branch
```

Select:

```
main
```

Choose folder:

```
/ root
```

Save.

GitHub will give you a website URL.

Simple meaning:

GitHub Pages takes your HTML project and publishes it online.

Part 9: Merge Conflicts

23. What Is a Merge Conflict?

A merge conflict happens when Git cannot automatically combine changes.

Example:

On `main`, a line says:

```
Hello from main
```

On your branch, the same line says:

```
Hello from feature
```

Git does not know which one is correct.

So Git asks you to fix it manually.

Simple meaning:

A merge conflict means Git needs human help.

24. Conflict Markers

When a conflict happens, Git shows special markers inside the file:

```
<<<<<< HEAD
This is the current branch change.
=====
This is the incoming branch change.
>>>>>> origin/main
```

Meaning:

```
<<<<<<< HEAD
```

Start of your current branch version.

```
=====
```

Divider between two versions.

```
>>>>>>> origin/main
```

End of incoming branch version.

Simple meaning:

Git shows both versions and asks you to choose what the final file should contain.

25. How to Resolve a Conflict

When you see this:

```
<<<<<<< HEAD
Hello from my branch
=====
Hello from main branch
>>>>>>> origin/main
```

You must edit the file manually.

You can keep one side:

```
Hello from my branch
```

Or keep the other side:

```
Hello from main branch
```

Or combine both:

```
Hello from my branch and main branch
```

Then remove the conflict markers.

After fixing:

```
git add .  
git commit -m "chore: resolve conflicts"
```

Simple meaning:

Fix the file, remove the markers, stage it, and commit the fix.

Part 10: Rebase Basics

26. What Is Rebase?

Rebase moves your branch commits on top of the latest main branch.

Simple idea:

Before rebase:

```
main:    A---B---C  
feature:  D---E
```

After rebase:

```
main:    A---B---C  
feature:          D---E
```

Simple meaning:

Rebase makes your branch look like it started from the latest main.

27. Interactive Rebase

Interactive rebase lets you edit your commit history.

Command:

```
git rebase -i main
```

You may see words like:

```
pick  
squash
```

Meaning:

```
pick
```

Keep this commit.

```
squash
```

Combine this commit with the previous commit.

Simple meaning:

Interactive rebase lets you clean your commits before sharing.

28. What Is Squash?

Squash means combining multiple commits into one commit.

Example:

Before squash:

```
commit 1: update  
commit 2: fix  
commit 3: final fix
```

After squash:

```
commit 1: feat: add login page
```

Simple meaning:

Squash turns messy small commits into one clean commit.

Beginner warning:

Do rebase and squash only on practice branches first.

Do not rewrite shared team history until you understand it well.

Part 11: Sprint 9 Practice Workflow

Use your Sprint 7 or Sprint 8 practice repository.

Practice A: Add `.gitignore`

```
git checkout main
git pull
echo ".env" > .gitignore
echo "node_modules/" >> .gitignore
echo "dist/" >> .gitignore
echo ".DS_STORE" >> .gitignore
echo "Thumbs.db" >> .gitignore
git status
git add .gitignore
git commit -m "chore: add gitignore"
git push
```

Meaning:

You created `.gitignore`, added common ignored files, committed it, and pushed it.

Practice B: Create a Simple Conflict

Step 1: On `main`

```
git checkout main
git pull
echo "Sprint 9 main version" > conflict.md
git add conflict.md
```

```
git commit -m "docs: add main conflict file"
git push
```

Step 2: Create a branch

```
git checkout -b feat/conflict
echo "Sprint 9 feature version" > conflict.md
git add conflict.md
git commit -m "docs: update conflict file on feature"
```

Step 3: Go back to `main` and change the same file

```
git checkout main
echo "Sprint 9 changed main version" > conflict.md
git add conflict.md
git commit -m "docs: update conflict file on main"
```

Step 4: Try to merge

```
git checkout feat/conflict
git merge main
```

Now Git may show a conflict.

Step 5: Open `conflict.md` and fix the markers

Step 6: Finish

```
git add .
git commit -m "chore: resolve conflict"
```

Simple meaning:

This teaches you how conflicts happen and how to fix them.

Part 12: Sprint 9 Command Summary

```
git remote add upstream <url>
```

Add the original repository.

```
git fetch upstream
```

Download latest upstream history.

```
git merge upstream/main
```

Merge original `main` into your local branch.

```
git reset --hard upstream/main
```

Force your branch to match `upstream/main`. Dangerous.

```
git push --force
```

Force overwrite remote history. Dangerous.

```
git fetch origin
```

Download latest `origin` history.

```
git merge origin/main
```

Merge remote `main` into current branch.

```
git add .
```

Stage resolved conflict files.

```
git commit -m "chore: resolve conflicts"
```

Save the conflict fix.

```
git rebase -i main
```

Start interactive rebase.

```
git rebase --continue
```

Continue rebase after resolving conflict.

Part 13: Common Beginner Mistakes

Mistake 1: Thinking a fork and a clone are the same

Wrong idea:

Fork = clone

Correct idea:

- **Fork** = a GitHub copy of someone else's repository
- **Clone** = downloading a repository to your local computer

Mistake 2: Forgetting which remote is which

Wrong idea:

- `origin` and `upstream` mean the same thing

Correct idea:

- `origin` is usually your fork or your main remote
- `upstream` is usually the original repository

Mistake 3: Using `git pull` without understanding what it does

Correct idea:

- `git fetch` downloads only
- `git pull` downloads and merges

Mistake 4: Thinking `.gitignore` removes already committed secrets

Wrong idea:

I added `.env` to `.gitignore`, so the old committed `.env` is gone.

Correct idea:

`.gitignore` helps prevent future tracking, but it does not automatically erase already committed history.

Mistake 5: Using `git reset --hard` carelessly

This can delete local work.

Mistake 6: Using `git push --force` carelessly

This can overwrite remote history and confuse teammates.

Mistake 7: Panic when a merge conflict appears

A conflict does not mean the project is broken forever.

It only means Git needs you to choose the correct final content.

Mistake 8: Not removing conflict markers after fixing a file

If you keep these markers in the file:

```
<<<<<<<
=====
>>>>>>>
```

then the file is still not cleaned properly.

Mistake 9: Using rebase on shared history too early

Rebase is powerful, but beginners should first practice it on personal or test branches.

Part 14: Sprint 9 Checkpoint

You pass Sprint 9 when you can do this:

Resolve one simple merge conflict and explain these markers:

```
<<<<<<
=====
>>>>>>
```

Good beginner answer:

```
<<<<<< shows the start of my current branch change.
===== separates my version from the incoming version.
>>>>>> shows the end of the incoming branch change.
```

Even simpler:

```
Git shows both versions. I choose the correct final version, remove the markers,
then commit the fix.
```

Part 15: Final Sprint 9 Understanding

After Sprint 9, you should understand:

- Forking is used when you do not have write access.
- `origin` usually means your fork or your main remote.
- `upstream` usually means the original repository.
- `fetch` downloads remote information without automatically merging.
- `pull` downloads and tries to merge.
- `.gitignore` prevents private or unnecessary files from being tracked.
- GitHub Pages publishes simple static websites.
- Merge conflicts happen when Git cannot safely combine changes.
- Conflict markers show the two versions that need manual fixing.
- Rebase moves your commits on top of another branch.
- Interactive rebase can clean commit history using `pick` and `squash`.

For now, focus mainly on:

- `.gitignore`
- syncing with `upstream`
- resolving one simple conflict

Rebase is important, but it is safer to practice it only on test branches first.

Part 16: Quick Revision Sheet

Key Terms

- **Fork** = your GitHub copy of another repository
- **Write access** = permission to push directly
- **origin** = your main remote or your fork
- **upstream** = the original repository
- **fetch** = download remote information only
- **pull** = fetch + merge
- **.gitignore** = file that tells Git what not to track
- **GitHub Pages** = static website hosting from a GitHub repo
- **Merge conflict** = Git cannot combine changes automatically
- **Conflict markers** = special lines Git adds during a conflict
- **Rebase** = move commits on top of newer history
- **Interactive rebase** = clean commit history manually
- **Squash** = combine commits into one

Most Important Sprint 9 Memory

Fork if you cannot push directly.
Use origin for your copy.
Use upstream for the original repo.
Use .gitignore to avoid tracking private files.
Fix conflicts by editing the file, removing markers, staging, and committing.

Safe Beginner Rule

Use these carefully:

```
git reset --hard  
git push --force  
git rebase -i main
```

These are powerful commands and can cause problems if used without understanding them.